

Reviewer Pack — Hessian-Rank of Clause-Count Polynomial Bounds DPLL Depth

Entry 67153fba470b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 22:23:58 UTC

Hessian-Rank of Clause-Count Polynomial Bounds DPLL Depth

Entry ID: 67153fba470b **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 22:23:58 UTC

1. Conjecture

Field A (mathematical branch): Geometric Complexity Theory: the Mignon-Ressayre Hessian-rank lower bound for determinantal complexity $dc(p)$ — rank of $Hess(p)$ at a smooth point lower-bounds $2 \cdot dc(p)$; the most computationally tractable orbit-closure obstruction in the Mulmuley-Sohoni / Bürgisser-Ikenmeyer lineage. **Field B** (complexity object): DPLL search-tree depth $d_{DPLL}(F)$ of unsatisfiable 3-CNF formulas F at clause density $\alpha=5.0$ (lex-DPLL with unit propagation), equivalently tree-resolution depth via the standard correspondence.

Statement:

For every UNSAT 3-CNF F on $n \in [6,11]$ variables at clause density $\alpha=5.0$, define $p_F(x) := \sum_{C \in F} \prod_{l \in C} (1 - l) \in Q[x_1, \dots, x_n]$, the degree-3 multilinear ‘unsatisfied-clause counter’ ($l := x_i$ for positive literal x_i , $l^* := 1 - x_i$ for negative). Let $H_F(x)$ be its $n \times n$ symmetric Hessian matrix of linear forms and $r(F) := \max_{x \in \{0,1,2\}^n} \text{rank}_Q(H_F(x))$. Then $r(F) \geq d_{DPLL}(F)$; a single F with $r(F) < d_{DPLL}(F)$ refutes the conjecture.

Rationale (proposer’s reasoning):

GCT lower bounds for determinantal/permanental orbit closures classically route through Hessian rank at a smooth point (Mignon-Ressayre 2004; Landsberg-Manivel-Ressayre), and that invariant is the canonical computable GCT witness. The clause-count polynomial p_F is the natural degree-3 GL_n -equivariant arithmetization of an UNSAT 3-CNF: its Hessian at integer evaluation points is a small integer matrix whose rank measures the ‘algebraic spread’ of clause-variable-pair couplings in F . Linking Mignon-Ressayre rank to DPLL depth would import GCT-style algebraic-geometry obstructions into proof complexity — a bridge essentially unexplored — while staying within concretely computable invariants.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ed66eacd778f61d8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all UNSAT 3-CNF instances generated (40 per $n \in \{6..11\}$, $\alpha=5.0$, kept if UNSAT), the conjecture is SUPPORTED iff $\geq 95\%$ of instances satisfy $r(F) \geq d_DPLL(F)$ AND no single instance exhibits $r(F) < d_DPLL(F)$; any single counterexample falsifies it.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Hessian rank determinantal complexity DPLL resolution depth 3-CNF - Mignon Ressayre lower bound SAT tree resolution polynomial-geometric complexity theory propositional proof complexity Hessian obstruction

Top relevant hits considered: - [<http://arxiv.org/abs/cs/0209032v3>] Complexity Results on DPLL and Resolution - [<http://arxiv.org/abs/1504.00151v1>] Bi-polynomial rank and determinantal complexity - [<http://arxiv.org/abs/1403.8106v1>] Recent advances on the log-rank conjecture in communication complexity - [<http://arxiv.org/abs/1002.1290v3>] Bounds on Threshold of Regular Random k -SAT - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/quant-ph/0305179v3>] Polynomial Degree and Lower Bounds in Quantum Complexity: Collision and Element Distinctness with Small Range - [<http://arxiv.org/abs/1611.00827v2>] Geometric complexity theory and matrix powering - [<http://arxiv.org/abs/0911.3482v5>] Complexity of Networks (reprise)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    rank = sum(1 for row in A if any(row))
```

```

    return rank

def lex_dp11(F, assignment):
    if not F:
        return True
    var = next(v for v in range(len(assignment)) if assignment[v] is None)
    for val in [0, 2]:
        new_assignment = assignment[:]
        new_assignment[var] = val
        if all(new_assignment[v] == (F[i][v] > 0) * 2 - 1 for i in range(len(F))):
            if lex_dp11(F, new_assignment):
                return True
    return False

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [6, 7, 8, 9, 10, 11]
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        alpha = 5.0
        F_count = 40
        d_DPLL_sum = 0

        for _ in range(F_count):
            variables = list(range(n))
            clauses = []
            for _ in range(int(alpha * n * (n - 1) / 6)):
                clause = random.sample(variables, 3)
                if random.choice([True, False]):
                    clause = [-v for v in clause]
                clauses.append(clause)

            F = [[0] * n for _ in range(n)]
            for clause in clauses:
                for l in clause:
                    i = abs(l) - 1
                    F[i][i] += (-1 if l < 0 else 1)

            d_DPLL_F = lex_dp11(F, [None] * n)
            d_DPLL_sum += d_DPLL_F

            H_F = [[0] * n for _ in range(n)]
            for i in range(n):
                for j in range(n):
                    H_F[i][j] = sum((-1 if l < 0 else 1) * (-1 if m < 0 else 1) for l, m in zip(F[i], F[j]))

            rank_F = gaussian_elimination(H_F)

            instances_tested += 1
            if rank_F < d_DPLL_F:
                conjecture_holds = False
                counterexample = f"n={n}, alpha=5.0, d_DPLL(F)={d_DPLL_F}, rank_F={rank_F}"

```

```

mean_d_DPLL = d_DPLL_sum / instances_tested
std_d_DPLL = math.sqrt(sum((d_DPLL - mean_d_DPLL) ** 2 for d_DPLL in range(d_DPLL_sum // insta

return {
    "metric_name": "DPLL Depth",
    "metric_value": mean_d_DPLL,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_d_DPLL = sum(r["metric_value"] for r in results) / len(results)
    std_d_DPLL = math.sqrt(sum((r["metric_value"] - mean_d_DPLL) ** 2 for r in results) / len(resu
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_d_DPLL} std={std_d_DPLL} support_fraction={support_fr
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.95:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cla6b4ba.py", line 104, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cla6b4ba.py", line 72, in run_trial
    d_DPLL_F = lex_dp11(F, [None] * n)
               ^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cla6b4ba.py", line 40, in lex_dp11
    if all(new_assignment[v] == (F[i][v] > 0) * 2 - 1 for i in range(len(F))):
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cla6b4ba.py", line 40, in <genexpr>
    if all(new_assignment[v] == (F[i][v] > 0) * 2 - 1 for i in range(len(F))):
    ^

```

NameError: name 'v' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `NameError` ('v' is not defined) in `lex_dpll` before any instances were evaluated, so no data was produced to assess the support or falsification condition.
| next: Fix the scoping bug in `lex_dpll` (the `genexpr` references undefined 'v'; likely should iterate over variables in the clause) and rerun the trial.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	302545
2	preregistration	claude_max	opus	0	0	5708
3	novelty	claude_max	opus	0	0	3066
4	novelty	claude_max	opus	0	0	8462
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19075
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12495
7	judge	claude_max	opus	0	0	3788

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 355140 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/67153fba470b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/67153fba470b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/67153fba470b.tar.gz` (if generated)